

Week 9: Java Threads and Concurrency

Overview

This week explores concurrent programming in Java, covering thread creation, synchronization, and modern concurrency utilities. Learn how to write programs that can execute multiple tasks simultaneously, improving performance and responsiveness.

Learning Objectives

By the end of this week, you should be able to:

- Understand thread lifecycle and states
- Create threads using multiple approaches
- Manage thread execution with control methods
- Identify and prevent race conditions
- Implement thread synchronization
- Use modern concurrency APIs (ExecutorService, CompletableFuture)
- Apply concurrency to real-world problems

Why Concurrency Matters

Real-world Applications:

- Web servers handling multiple requests simultaneously
- GUI applications keeping interfaces responsive
- Data processing in parallel
- AI/ML model training
- Game development (graphics, physics, AI on separate threads)

Benefits:

- Improved responsiveness
- Better resource utilization (multi-core CPUs)
- Parallel processing of independent tasks
- Enhanced user experience

Example Index

Getting Started

Start Here: Run `Week09ExampleIndex.java` to see all available examples and how to run them.

```
mvn exec:java -Dexec.mainClass="com.example.basic.week09Concurrency.Week09ExampleIndex"
```

Basic Thread Concepts (Examples 1-5)

Example 1: Thread Lifecycle

File: `Example01ThreadLifecycle.java`

What It Demonstrates:

- Thread states: NEW → RUNNABLE → TIMED_WAITING → TERMINATED
- `Thread.getState()` for monitoring thread status
- `Thread.sleep()` for timed waiting
- `join()` to wait for thread completion

Key Concepts:

```
Thread t = new Thread(() -> { /* work */ });
t.getState(); // NEW
t.start();    // NEW → RUNNABLE
t.join();     // Wait for completion
t.getState(); // TERMINATED
```

How to Run:

```
mvn exec:java -Dexec.mainClass="com.example.basic.week09Concurrency.Example01ThreadLifecycle"
```

Example 2: Extending Thread Class

File: `Example02CounterThread.java`

What It Demonstrates:

- Method 1: Creating threads by extending Thread class
- Overriding run() method
- Starting multiple threads concurrently
- Thread naming for debugging

Advantages: Direct control, simple for single-purpose threads

Disadvantages: Can't extend other classes (Java single inheritance)

Key Code Pattern:

```
public class MyThread extends Thread {  
    @Override  
    public void run() {  
        // Thread work here  
    }  
}  
MyThread t = new MyThread();  
t.start();
```

How to Run:

```
mvn exec:java -Dexec.mainClass="com.example.basic.week09Concurrency.Example02CounterThread"
```

Example 3: Implementing Runnable

File: Example03TaskRunner.java

What It Demonstrates:

- Method 2: Implementing Runnable interface
- Separation of task logic from thread management
- Passing Runnable to Thread constructor
- Better OOP design with composition

Advantages: Can implement multiple interfaces, reusable task logic

Disadvantages: Slightly more verbose

Key Code Pattern:

```
public class MyTask implements Runnable {  
    public void run() {  
        // Task logic here  
    }  
}  
  
Thread t = new Thread(new MyTask(), "ThreadName");  
t.start();
```

How to Run:

```
mvn exec:java -Dexec.mainClass="com.example.basic.week09Concurrency.Example03TaskRunner"
```

Example 4: Modern Thread Creation

File: Example04ModernThreads.java

What It Demonstrates:

- Method 3: Lambda expressions for threads (Java 8+)
- Method references as Runnable
- Thread factory patterns
- Concise, modern code style

Key Code Patterns:

```
// Lambda expression  
Thread t1 = new Thread(() -> { /* work */ });  
  
// Method reference  
Thread t2 = new Thread(MyClass::myMethod);  
  
// Factory method  
Thread t3 = createWorkerThread("Task", 5);
```

How to Run:

```
mvn exec:java -Dexec.mainClass="com.example.basic.week09Concurrency.Example04ModernThreads"
```

Example 5: Thread Control Methods

File: Example05ThreadControl.java

What It Demonstrates:

- start() - Begin thread execution
- join() / join(timeout) - Wait for completion
- sleep() - Pause execution
- interrupt() - Request thread stop
- isAlive() - Check if running
- yield() - Suggest scheduler switch
- setPriority() - Set execution priority

Important Methods:

```
thread.start();           // Start execution
thread.join();            // Wait forever
thread.join(5000);        // Wait max 5 seconds
thread.interrupt();       // Request interruption
boolean alive = thread.isAlive();
thread.setPriority(Thread.MAX_PRIORITY);
```

How to Run:

```
mvn exec:java -Dexec.mainClass="com.example.basic.week09Concurrency.Example05ThreadControl"
```

Synchronization (Examples 6-8)

Example 6: Race Condition Problem

File: Example06RaceCondition.java

What It Demonstrates:

- THE PROBLEM: Race conditions with shared data
- Data corruption from unsynchronized access
- Lost updates when multiple threads modify shared variables

- Why synchronization is necessary

The Problem:

```
private static int counter = 0;

public static void unsafeIncrement() {
    counter++; // NOT ATOMIC! Read-Modify-Write
}

// 10 threads × 1000 increments = Expected: 10,000
// Actual result: < 10,000 (data loss!)
```

Expected vs Actual:

- Expected: 10,000 increments
- Actual: Usually 7,000-9,000 (varies each run)
- Lost updates due to race conditions

How to Run:

```
mvn exec:java -Dexec.mainClass="com.example.basic.week09Concurrency.Example06RaceCondition"
```

Example 7: Synchronization Solutions

File: Example07SafeCounter.java

What It Demonstrates:

- Solution 1: Synchronized methods
- Solution 2: Synchronized blocks with explicit locks
- Solution 3: AtomicInteger (lock-free)
- Comparing performance and use cases

Three Approaches:

1. Synchronized Method:

```
public synchronized void increment() {  
    count++; // Only one thread at a time  
}
```

2. Synchronized Block:

```
private final Object lock = new Object();  
  
public void deposit(double amount) {  
    synchronized(lock) {  
        balance += amount; // Critical section  
    }  
}
```

3. Atomic Variables:

```
private AtomicInteger count = new AtomicInteger(0);  
  
public void increment() {  
    count.incrementAndGet(); // Lock-free, atomic  
}
```

When to Use What:

- Synchronized methods: Simple, protects entire method
- Synchronized blocks: Fine-grained control, better performance
- Atomic variables: Best performance for simple operations

How to Run:

```
mvn exec:java -Dexec.mainClass="com.example.basic.week09Concurrency.Example07SafeCounter"
```

Example 8: Producer-Consumer Pattern

File: Example08ProducerConsumer.java

What It Demonstrates:

- Classic concurrency pattern for thread coordination
- wait() and notifyAll() for inter-thread communication
- Bounded buffer with capacity limits
- One producer feeding multiple consumers

Key Synchronization:

```
synchronized (lock) {  
    while (buffer.isFull()) {  
        lock.wait(); // Wait for space  
    }  
    buffer.add(item);  
    lock.notifyAll(); // Wake consumers  
}
```

Pattern Elements:

- Producer thread creates items
- Consumer threads process items
- Shared buffer with size limit
- wait() when buffer full/empty
- notifyAll() to wake waiting threads

How to Run:

```
mvn exec:java -Dexec.mainClass="com.example.basic.week09Concurrency.Example08ProducerConsumer"
```

Modern Concurrency (Examples 9-10)

Example 9: ExecutorService & CompletableFuture

File: Example09ExecutorService.java

What It Demonstrates:

- Modern thread pool management
- ExecutorService for task submission
- Callable for tasks that return values
- Future for retrieving results

- CompletableFuture for async pipelines

Three Demonstrations:

1. Thread Pool:

```
ExecutorService executor = Executors.newFixedThreadPool(3);
executor.submit(() -> { /* task */ });
executor.shutdown();
```

2. Callable and Future:

```
Callable<Integer> task = () -> 42 * 2;
Future<Integer> future = executor.submit(task);
Integer result = future.get(); // Blocks until ready
```

3. CompletableFuture:

```
CompletableFuture.supplyAsync(() -> "data")
    .thenApply(String::toUpperCase)
    .thenApply(s -> s + " PROCESSED")
    .get(); // Returns processed result
```

Benefits Over Manual Threads:

- Thread reuse (better performance)
- Easy task submission
- Built-in shutdown management
- Return values from tasks
- Async composition with CompletableFuture

How to Run:

```
mvn exec:java -Dexec.mainClass="com.example.basic.week09Concurrency.Example09ExecutorService"
```

Example 10: Parallel Text Processing (AI Example)

File: Example10ParallelTextProcessor.java

What It Demonstrates:

- Real-world AI application: text analysis
- Parallel processing across CPU cores
- Sentiment analysis (positive/negative/neutral)
- Topic extraction from documents
- Performance benefits of concurrency

What It Does:

1. Analyzes 5 text documents in parallel
2. Performs sentiment analysis on each
3. Extracts top topics (frequent words)
4. Uses CPU cores efficiently
5. Aggregates and reports results

AI Features:

- Sentiment analysis (positive/negative words)
- Topic extraction (word frequency)
- Stop word filtering
- Parallel processing for speed

Performance:

- Sequential: ~15 seconds (5 docs × 3 sec each)
- Parallel: ~3-4 seconds (using multiple cores)
- **3-4x speedup** from concurrency

Key Code:

```
int numThreads = Runtime.getRuntime().availableProcessors();
ExecutorService executor = Executors.newFixedThreadPool(numThreads);

// Submit analysis tasks
for (String text : texts) {
    Future<Result> future = executor.submit(new AnalysisTask(text));
    futures.add(future);
}

// Collect results
for (Future<Result> future : futures) {
    Result r = future.get(); // Wait for this task
    System.out.println(r);
}
```

How to Run:

```
mvn exec:java -Dexec.mainClass="com.example.basic.week09Concurrency.Example10ParallelTextProces;
```

Quick Reference Guide

Thread Creation Methods

Method	Code	Pros	Cons
Extend Thread	class MyThread extends Thread	Simple, direct control	Single inheritance limit
Implement Runnable	class Task implements Runnable	Flexible, reusable	More verbose
Lambda	new Thread(() -> {...})	Concise, modern	None

Thread Control Methods

Method	Purpose	Example
start()	Begin execution	thread.start()

Method	Purpose	Example
join()	Wait for completion	thread.join()
sleep(ms)	Pause thread	Thread.sleep(1000)
interrupt()	Request stop	thread.interrupt()
isAlive()	Check if running	if (thread.isAlive())

Synchronization Techniques

Technique	Use Case	Performance
Synchronized methods	Entire method protection	Medium
Synchronized blocks	Fine-grained control	Better
Atomic variables	Simple atomic operations	Best
wait()/notify()	Thread coordination	Special purpose

Modern Concurrency APIs

API	Purpose	Best For
ExecutorService	Thread pool	Multiple tasks
Callable	Tasks with return values	Computations
Future	Async result retrieval	Waiting for results
CompletableFuture	Async pipelines	Complex workflows

Common Pitfalls and Solutions

Pitfall 1: Race Conditions

Problem: Multiple threads accessing shared data without synchronization

Solution: Use synchronized, locks, or atomic variables

Pitfall 2: Deadlock

Problem: Two threads waiting for each other's locks

Solution: Always acquire locks in the same order

Pitfall 3: Forgetting to Start Thread

Problem: Creating thread but not calling start()

Solution: Always call `thread.start()` , not `thread.run()`

Pitfall 4: Not Closing ExecutorService

Problem: Application doesn't exit, threads keep running

Solution: Always call `executor.shutdown()`

Pitfall 5: Ignoring InterruptedException

Problem: Swallowing exceptions, thread won't stop properly

Solution: Restore interrupt status: `Thread.currentThread().interrupt()`

Practice Exercises

Exercise 1: Multi-threaded Download Simulator

Create a program that simulates downloading multiple files concurrently.

- Use ExecutorService with fixed thread pool
- Each "download" takes random time (1-5 seconds)
- Track and display progress
- Report total time vs sequential time

Exercise 2: Thread-Safe Bank Account

Implement a thread-safe bank account class:

- Methods: `deposit()`, `withdraw()`, `getBalance()`
- Test with multiple threads doing random transactions
- Verify balance is always correct
- Compare synchronized vs atomic approaches

Exercise 3: Parallel File Word Counter

Build a parallel word counter for text files:

- Scan directory for .txt files
- Process files concurrently
- Count total words across all files
- Find most common words globally
- Report processing time

Exercise 4: Web Crawler Simulator

Simulate a multi-threaded web crawler:

- BlockingQueue for URLs to process
- Multiple worker threads
- Track visited URLs (thread-safe Set)
- Implement politeness delay
- Display crawling statistics

Best Practices

1. **Use thread pools** instead of creating threads manually
2. **Prefer immutable objects** for shared data when possible
3. **Always close ExecutorService** to prevent thread leaks
4. **Handle interruptions properly** - don't swallow InterruptedException
5. **Test with multiple threads** - race conditions may not appear in single-threaded tests
6. **Use concurrent collections** (ConcurrentHashMap, BlockingQueue) for shared data structures
7. **Keep synchronized blocks small** - only protect critical sections
8. **Document thread-safety** - clearly mark which methods are thread-safe

Performance Considerations

Thread Pool Sizing:

- CPU-intensive tasks: Number of cores + 1
- I/O-intensive tasks: Number of cores × 2 (or more)
- Use `Runtime.getRuntime().availableProcessors()` to get core count

When to Use Concurrency:

- ✓ Independent tasks that can run in parallel
- ✓ I/O operations (network, disk)
- ✓ Long-running computations
- ✗ Tasks with heavy shared data (synchronization overhead)
- ✗ Very quick operations (thread overhead exceeds benefit)

Next Steps

After mastering Week 9 concepts:

- Learn about advanced synchronization (ReadWriteLock, Semaphore)
- Explore parallel streams in Java
- Study concurrent design patterns
- Practice with real-world applications
- Understand the Java Memory Model

Additional Resources

- Java Concurrency Tutorial: [Oracle Docs](#)
- "Java Concurrency in Practice" by Brian Goetz (recommended book)
- Each example file contains detailed comments and explanations
- enhanced_week_9.md for complete lecture slides